

Overview

Algorithm Analysis

44

Analysis of Algorithm

Objective

The purpose of algorithm analysis is to determine:

- **Time efficiency**
Performance in terms of running times for different input sizes
- **Space utilization**
Requirement of auxiliary storage to run the algorithm
- **Correctness of algorithm**
Results are trustworthy, and algorithm is robust

45

Algorithm Analysis

Approaches

Basically three approaches can be adopted to analyze algorithm *running time* in terms of *input size*:

Empirical Approach

Running time measured experimentally

Analytical Approach

Running time estimated using mathematical modeling

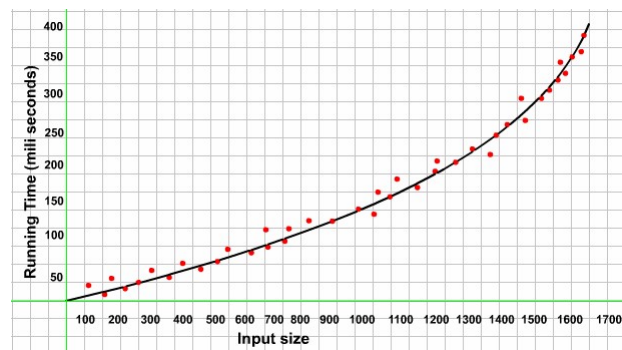
Visualization

Performance is studied through animation for different data sets

46

Empirical Analysis

The running time of algorithm is measured for different data sizes and time estimates are plotted against the input. The graph shows the trend.



47

Empirical Analysis

Limitations

Running time critically depends on:

- **Hardware resources used**
(CPU speed, IO throughput, RAM size etc.)
- **Software environment**
(Compiler, Programming Language etc.)
- **Program design approach**
(Conventional, Structured, Object Oriented)

48

Analytical Approach

Strategy: Running time is estimated by analyzing the *primitive operations* which make **significant contributions** to the overall algorithm time. These may broadly include:

- *Assigning a value*
- *Comparing data items*
- *Computing a value*
- *Moving a data item*
- *Calling a procedure*
- *Returning a value*

49

Algorithm Specification

Conventions

- *Plain natural language*
High level description
- *Pseudo Code*
Low level to facilitate analysis and implementation

50

Specification Using Natural Language

Preorder Tree Traversal Algorithm

- Step1.** *Push tree root to stack*
- Step2.** *Pop the stack. If stack is empty exit, else process the node*
- Step3.** *Travel down the tree following the left most path, and pushing each right child onto to the stack*
- Step4.** *When leaf node is reached, go back to step 2.*

51

Pseudo Code Convention

Declaration: Algorithm procedure name with parameters e.g MERGE(A,p,q)

Assignments : Using left arrows ($\leftarrow$) e.g $j \leftarrow k \leftarrow p$

Comparisons : Using symbols $\leq \geq \neq = > <$

Logical : Using connectives **and , or**

Computations: Using arithmetic symbols

Exchanges: Using symbol $\leftrightarrow$ e.g $A[i] \leftrightarrow A[k]$ (swap)

Loops:

for ---- **do** ----, **for**-----**downto** ----**do**---

while ---- **do**---

do ---- **until**---

Conditions: **if** ---- **then**-----**else**-----

Block structure: Using indentation

do---

do----

if--- **else** ----

Comments: Using symbol $\blacktriangleright$

52

Pseudo Code Convention

Example

```
1 for j  $\leftarrow$  2 to n
2   do key  $\leftarrow$  A[j]
3  $\blacktriangleright$  Insert A[j] into sorted sequence A[1..j-1]
4   i  $\leftarrow$  j - 1
5   while i > 0 and A[i] > key
6     do A[i+1]  $\leftarrow$  A[i]
7     i  $\leftarrow$  i-1
8   A[i+1]  $\leftarrow$  key
```

53

Types of Analysis

Best Case Analysis

- *Determines minimum possible running time*
- *Uses most favorable input of given size*
- *Little use in practice*

Worst Case Analysis

- *Determines maximum possible running time*
- *Uses most unfavorable input of a given size*
- *Most desirable in practice*

Average Case Analysis

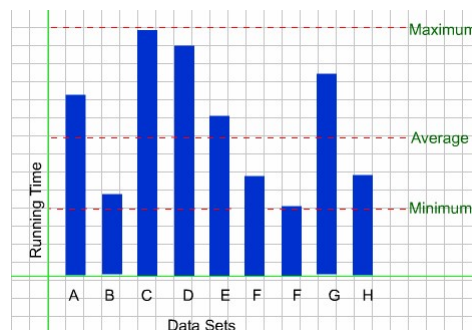
- *Determines average (expected) running time*
- *Uses probabilistic approach*
- *Difficult*

54

Empirical Study

Best, Average, Worst Cases Analysis

Empirical analysis can be used to estimate running times for different scenarios. The running time is measured for different data sets of the **same** size. A plot provides an indication **bounds** for maximum (best), average, and minimum (worst) running times.



55

Running Time Analysis

Linear search for value 25

Input Data Scenarios

Best case:	25	19	33	20	50	81	29	53	70	66
Average case:	70	19	81	20	29	25	50	66	33	53
Worst case:	70	20	66	33	50	81	53	19	29	25
Worst case (value non-existent)	53	70	66	20	50	81	29	33	19	16

56

Case Study

Problem

(1) Develop an algorithm to find maximum element in an array of size n

(2) Analyze the algorithm for:

- Time efficiency
- Space efficiency
- Correctness

57

Case Study

Algorithm

Step #1: Store first element of the array in variable *max*

Step #2: Scan array to compare *max* with other elements

Step #3: Replace *max* with a larger element

Step #4: Return value held by *max*

58

Case Study

Pseudo Code

The function **FIND-MAX** finds maximum element in an array. The array **A**, of size **n**, is passed as argument to the function.

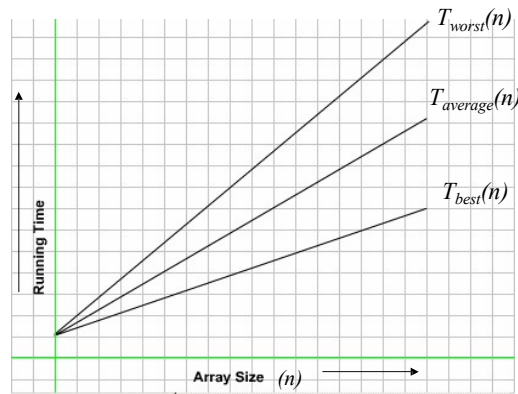
FIND-MAX (**A**[1..n])

1	<i>max</i> ← <i>A</i> [1]	► Store first array element into variable <i>max</i>
2	for <i>j</i> ← 2 to <i>n</i> do	► Scan remaining elements
3	if (<i>A</i> [<i>j</i>] > <i>max</i>)	► Compare an element with <i>max</i>
4	then <i>max</i> ← <i>A</i> [<i>j</i>]	► Replace <i>max</i> with a larger element
5	return <i>max</i>	► Return maximum element

59

Time Analysis

Comparison of Running Times



60

Case Study

Space Efficiency

➤Space analysis of algorithm for finding maximum element is simple and straight. It amounts to determining space utilization as function of array size.

- The total space requirement consists of memory used by the program statements and array element. The former is a fixed and does not depend on array size.
- The amount of storage requirement for the array depends on the nature of data type (integer, floating point, strings). It increases in direct proportion to array size.

Thus, if $S(n)$ is storage efficiency

$$S(n) = \underbrace{A}_{\text{Program space requirement}} + \underbrace{B.n}_{\text{Array space requirement}}$$

tar

61

Algorithm Analysis

Visualization

- Algorithm **animation** and visualization techniques are used to study
 - *Inner working and operation of an algorithm*
 - *Obtain statistics about the performance*
 - *Compare performance of competing algorithms*